Бібліотека для машинного навчання

scikit learn Install User Guide API Examples Community More Go

scikit-learn

Machine Learning in Python

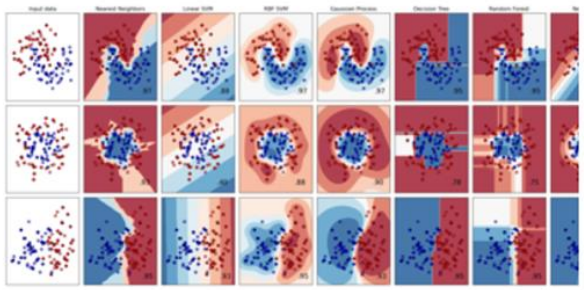
Getting Started | Release Highlights for 1.4 | GitHub

- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

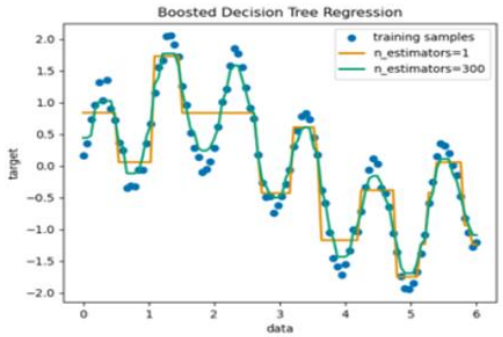
Applications: Spam detection, image recognition.
Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more...



Regression

Predicting a continuous-valued attribute associated with an object.

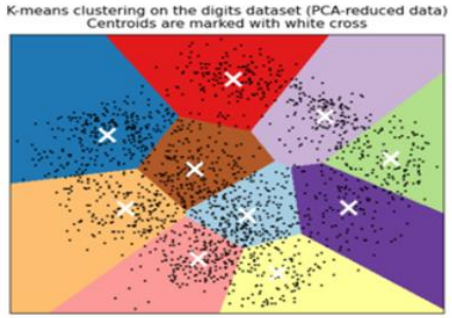
Applications: Drug response, Stock prices.
Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...



Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...

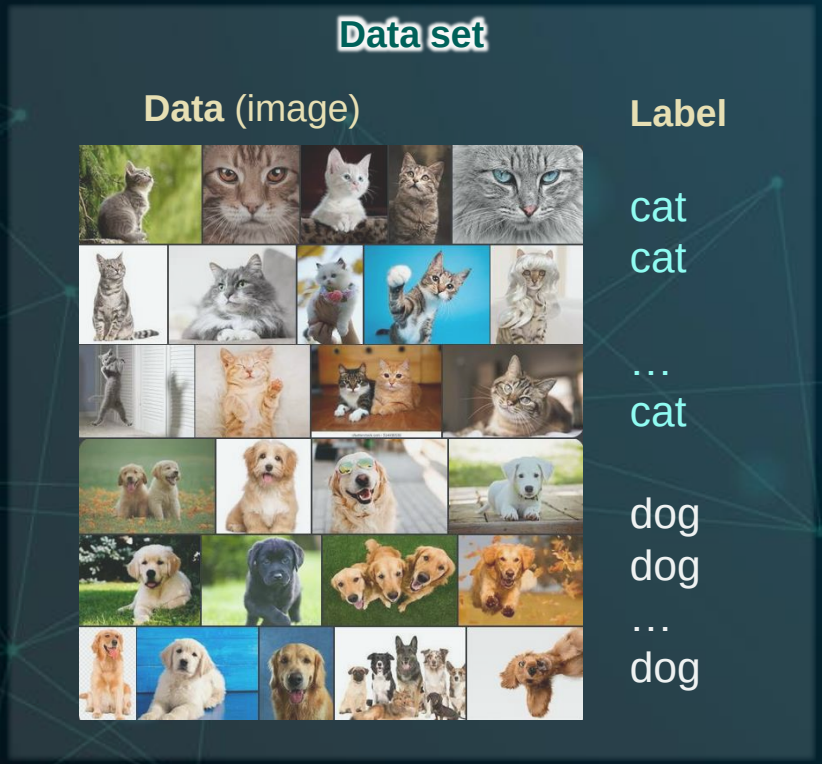


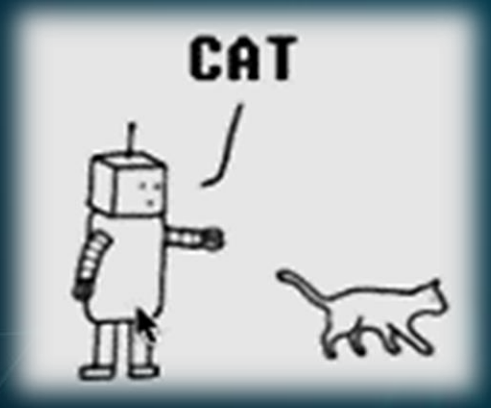
Навчання з вчителем (Supervised learning)

Алгоритми «з вчителем» навчаються з позначених навчальних даних, тобто для кожного навчального прикладу їм «відома» правильна відповідь.

Якщо алгоритм отримує набір зображень, позначених як собака або кішка, він може спочатку вгадати об’єкт на зображенні, а потім використати мітку, щоб перевірити, чи його припущення правильне.

Класифікація - найпопулярніша задача у всьому машинному навчанні.





Популярні алгоритми:

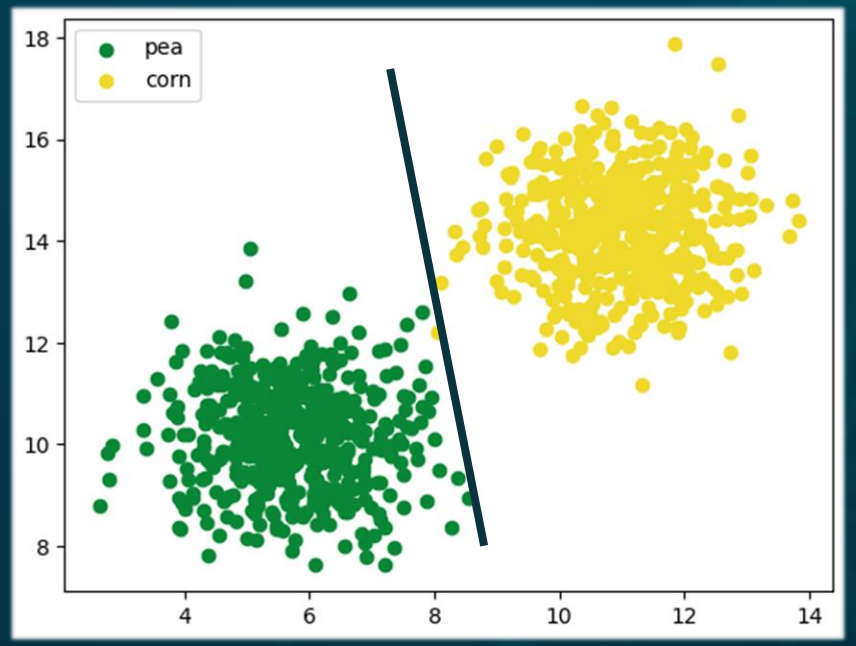
- Наївний Баєс (Naive Bayes),
- Дерева Рішень (Decision Trees),
- Логістична Регресія (Logistic Regression),
- K-найближчих сусідів (k-nearest neighbor),
- Метод Опорних Векторів (SVM).

Сьогодні для класифікації все частіше використовують нейромережі. Складніші дані - складніший алгоритм. Для простих даних із зрозумілими ознаками можна спробувати класичні алгоритми.
Для зображень, відео та іншої бігдати – є нейромережі.

Машинне навчання: класифікація

Якщо візуалізувати роботу переважної більшості алгоритмів машинного навчання, то це виглядатиме як побудова ліній крізь дані.

Припустимо, є завдання: класифікувати зображення кукурудзи та гороху.



З кожного зображення можна зробити висновок про колір і розмір об’єкту, і класифікувати їх як кукурудзу або горох.

Першим кроком буде отримання позначених навчальних даних. Тобто, отримання великої кількості зображень, кожне з яких позначено як кукурудза або горох.

З них отримати інформацію про колір і розмір, а потім побачити, як вони співвідносяться з цільовими класами.

Основні типи задач класифікації:

- Бінарна (Binary Classification)
- Багатокласова (Multi-Class Classification)
- З кількома мітками (Multi-Label Classification)
- Незбалансована (Imbalanced Classification)

Задача про фрукти: вирішення простої проблеми класифікації

Набір даних розробив д-р Ієн Мюррей
(Dr. Iain Murray) з Единбурзького університету

https://github.com/susanli2016/Machine-Learning-with-Python/blob/master/fruit_data_with_colors.txt

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
```

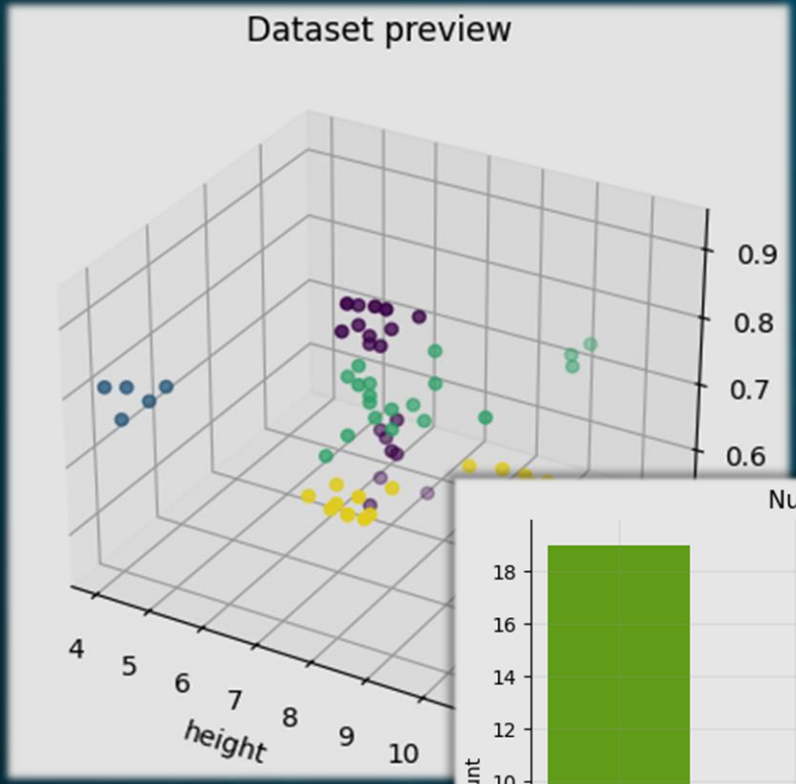
✓ 0.0s

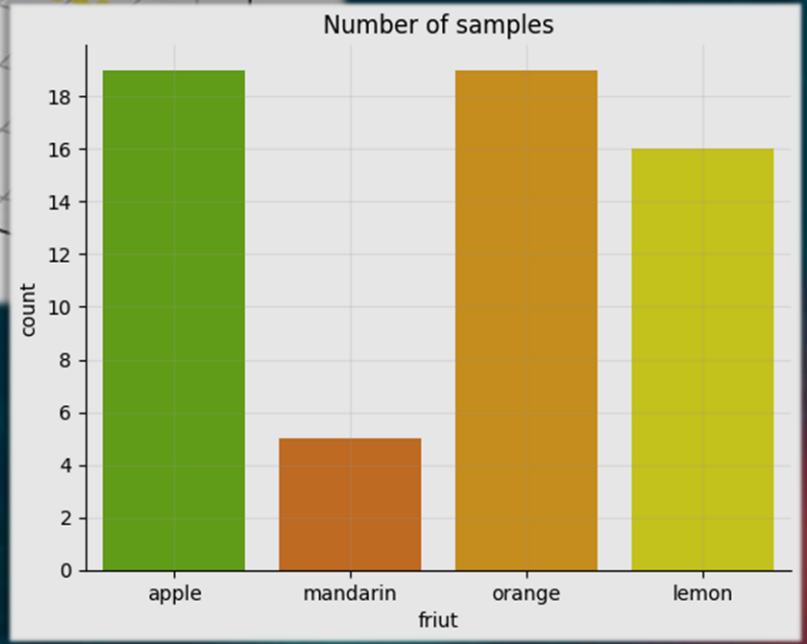
```
Fruits Dataset
```

```python
fruits = pd.read_table('fruit_data_with_colors.txt')
fruits.head(7)
```

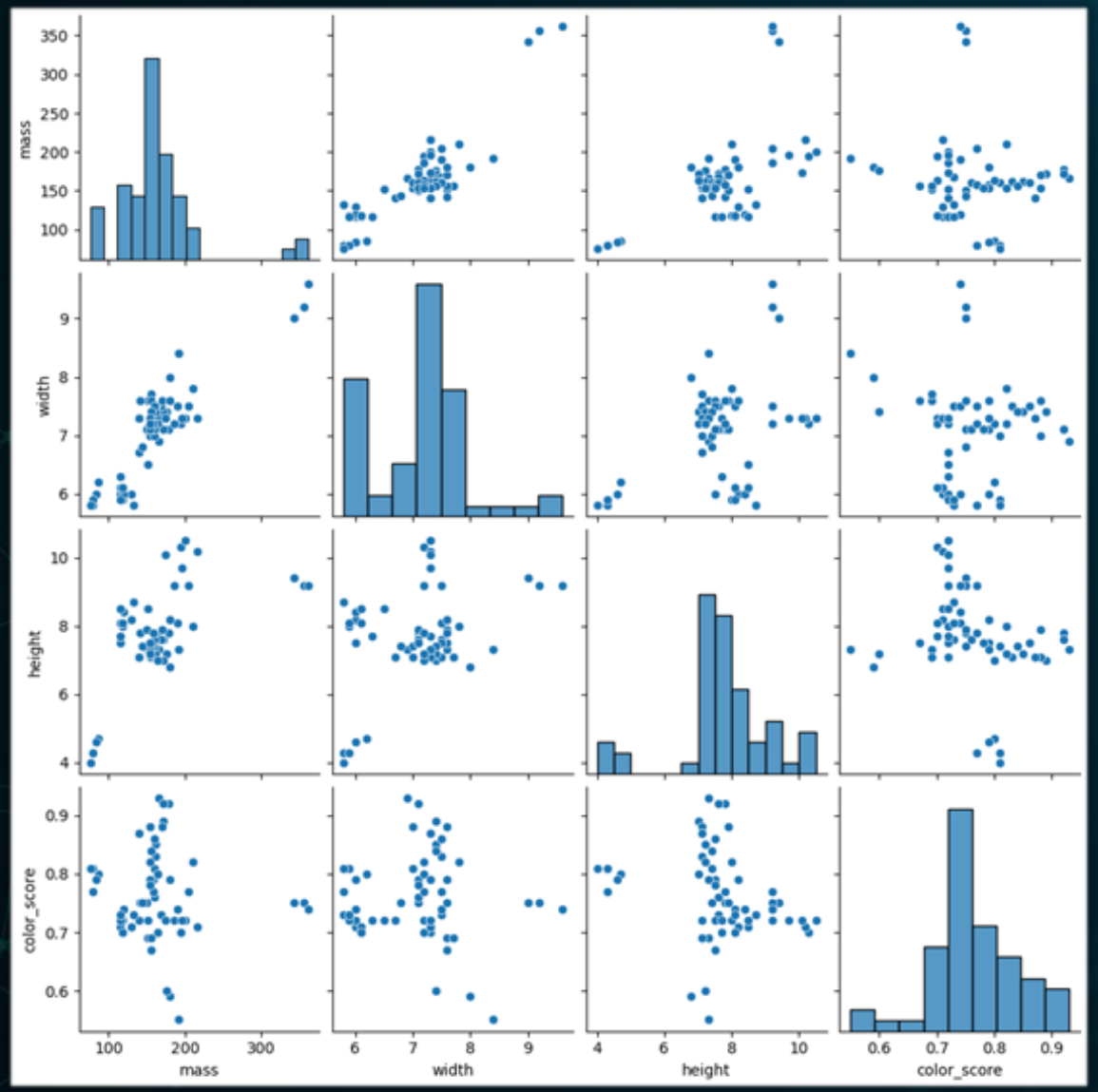
✓ 0.0s

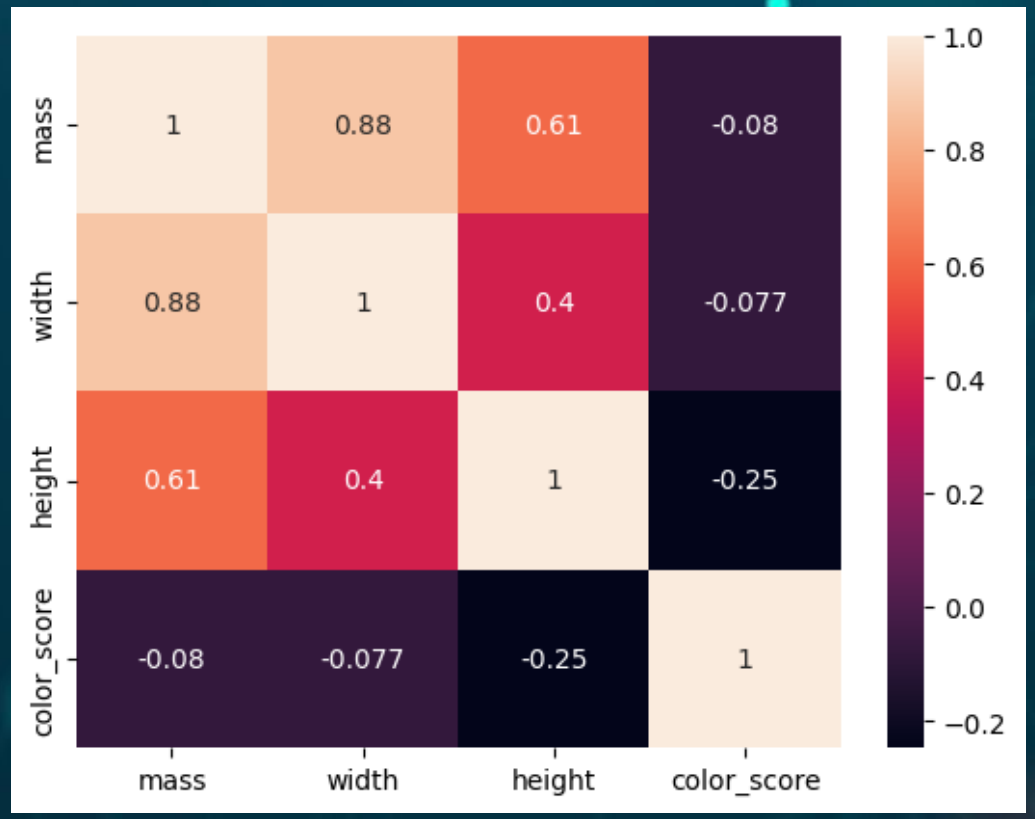
	fruit_label	fruit_name	fruit_subtype	mass	width	height	color_score
0	1	apple	granny_smith	192	8.4	7.3	0.55
1	1	apple	granny_smith	180	8.0	6.8	0.59
2	1	apple	granny_smith	176	7.4	7.2	0.60
3	2	mandarin	mandarin	86	6.2	4.7	0.80
4	2	mandarin	mandarin	84	6.0	4.6	0.79
5	2	mandarin	mandarin	80	5.8	4.3	0.77
6	2	mandarin	mandarin	80	5.9	4.3	0.81





Задача про фрукти: кореляція даних





Задача про фрукти: підготовка вхідних даних

Classification

```
from sklearn.model_selection import train_test_split

features_to_keep=['mass','width','height','color_score']

X=fruits[features_to_keep]
y=fruits['fruit_label']

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.33)

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

✓ 0.0s

```
fruits.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 59 entries, 0 to 58
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   fruit_label    59 non-null     int64
 1   fruit_name     59 non-null     object
 2   fruit_subtype  59 non-null     object
 3   mass           59 non-null     int64
 4   width          59 non-null     float64
 5   height         59 non-null     float64
 6   color_score    59 non-null     float64
dtypes: int64(3), object(2)
memory usage: 3.4+ KB
```

1. Розбити набір даних на навчальні дані та мітки до них

2. Розбити дані та тренувальний та тестовий набори

Train_test_split розбиває масиви чи матриці на випадкові тренувальні підмножини та тестові підмножини.

3. Нормалізувати дані

MinMaxScaler Трансформує дані, масштабуючи кожну ознаку окремо так, щоб вона була в заданому діапазоні на навчальному наборі, наприклад, між нулем і одиницею.

$$X_{std} = \frac{x - min}{max - min}$$

$$X_{scaled} = X_{std} * (max - min) + min$$

Задача про фрукти: навчання

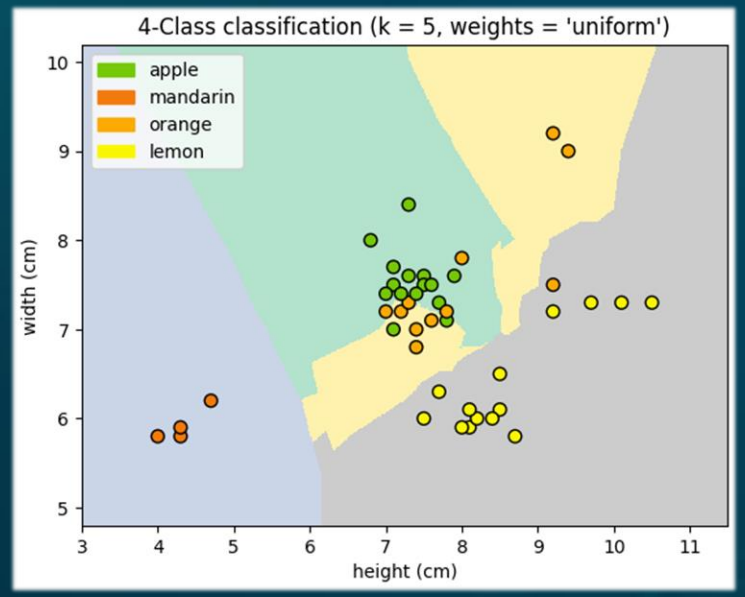
```python
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier()
knn.fit(X_train, y_train)

print('Accuracy of K-NN classifier on training set: {:.2f}'
     .format(knn.score(X_train, y_train)))
print('Accuracy of K-NN classifier on test set: {:.2f}'
     .format(knn.score(X_test, y_test)))
```

✓ 0.0s

```
Accuracy of K-NN classifier on training set: 0.95
Accuracy of K-NN classifier on test set: 0.95
```




```python
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

y_pred = knn.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

cm_display = ConfusionMatrixDisplay(cm).plot()
print(classification_report(y_test, y_pred))
```

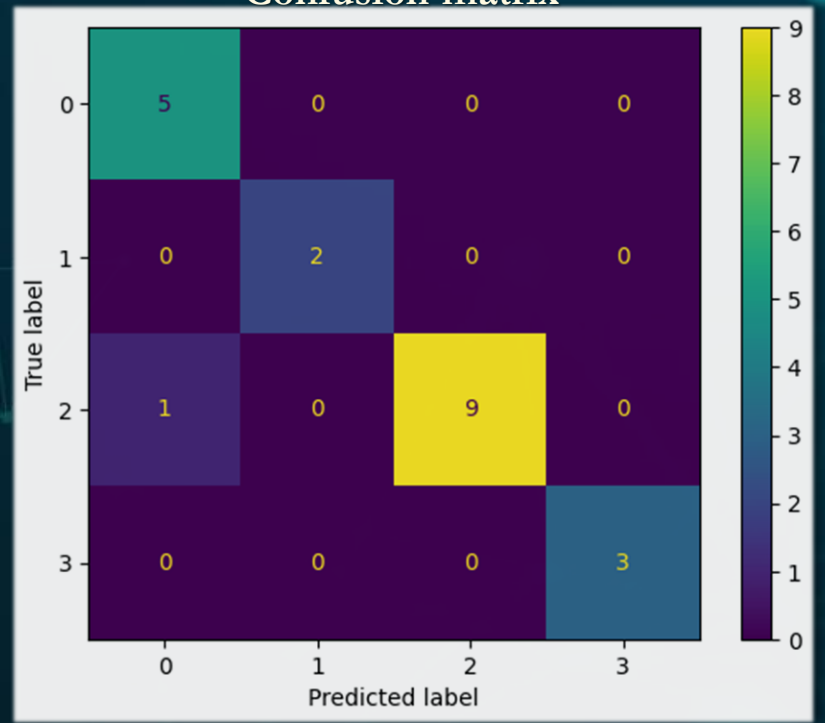
✓ 0.1s

```
              precision    recall  f1-score   support

           1       0.83      1.00      0.91         5
           2       1.00      1.00      1.00         2
           3       1.00      0.90      0.95        10
           4       1.00      1.00      1.00         3

    accuracy                           0.95        20
   macro avg       0.96      0.97      0.96        20
weighted avg       0.96      0.95      0.95        20
```

Матриця неточностей
Confusion matrix



Відображає кількість помилок при визначенні класів.
Рядок для кожного класу містить чарунки зі значення, що відображують скільки разів елементи цього класу плутали з елементами класу певного стовбця.

Для формування матриці неточності потрібно мати набір прогнозів щоб порівнювати їх з цілями.

Кожний рядок матриці неточностей представляє цільовий клас, а кожний стовпець – прогнозований клас.

Матриця неточностей, пояснення

Припустимо, що є два класи y = {0,1} і алгоритм, який передбачає приналежність кожного об'єкта одному з цих класів.

Розглянемо приклад. Нехай система захисту мережі використовує алгоритм класифікації для виявлення атаки: нормальна робота мережі чи аномальна (наявність атаки). При цьому у першому випадку система і далі нормально працює (y = 0) , а у другому – видається сигнал тривоги (y = 1) , що повідомляє про можливі ризики.

Для такого класифікатора може бути два види помилок:

1) нормальна ситуація у мережі за даними трафіка розпізнається моделлю як аномальна (помилкова тривога);
2) аномальна ситуація розпізнається як нормальна і ніяких дій щодо захисту від атаки не відбувається. Даний випадок можна розглядати як "пропуск цілі".

У разі "помилкової тривоги" втрати складуть тільки марно потрачений час та ресурси на протидію неіснуючій загрозі. У разі "пропуску цілі" можна втратити набагато більше (інформацію, роботу мережі та інше). Тому системі захисту важливіше не допустити "пропуск цілі", ніж "помилкову тривогу".

Оскільки з точки зору логіки завдання виявлення аномалій важливіше правильно розпізнати аномалію (атаку) з міткою y = 1, ніж помилитися в розпізнаванні нормальної роботи мережі, будемо називати відповідний результат класифікації позитивним (аномалія чи атака виявлені вірно), а протилежний - негативним (аномалії чи атаки немає y = 0).

Тоді можливі наступні чотири результати класифікації:

True Positive (TP) – позитивний клас розпізнано як позитивний (*наявність атаки класифікована як наявна атака*). Спостереження, для яких це має місце називаються істинно-позитивними.

True Negative (TN) – негативний клас розпізнано як негативний (*нормальна робота мережі класифікована як нормальна робота без аномалій*). Спостереження, яких це має місце, називаються істинно негативними.

False Positive (FP) – негативний клас був розпізнаний як позитивний (*помилка, в результаті якої нормальна робота мережі класифікована як аномальна*). Спостереження, для яких було отримано такий результат класифікації, називаються помилково-позитивними.

False Negative (FN) – позитивний клас був розпізнаний як негативний (*помилка, в результаті якої атака чи аномальна робота мережі розпізнана як нормальна*). Спостереження, для яких було отримано такий результат класифікації, називаються помилково-негативними.

Прогнозовані класи

Цільові класи	0	1
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 **TN**	1 1 1 1 А мали бути **0** ! **FP**
1	0 0 0 0 0 0 А мали бути **1**! **FN**	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 **TP**

Інший клас прийняли за 1

1 прийняли за інший клас

Деякі метрики

1. Акуратність (**Accuracy**).
Акуратністю називається пропорція точних прогнозів по відношенню до загальної кількості прогнозів, тобто це ймовірність того, що клас буде розпізнано правильно

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Припустимо, потрібно оцінити роботу спам-фільтра пошти.
У нас є 100 НЕ-спам листів, 90 з яких класифікатор визначив вірно (TN=90, FP= 10), і 10 спам-листів, 5 з яких класифікатор також визначив вірно (TP = 5, FN = 5).

$$Accuracy = \frac{5 + 90}{5 + 90 + 10 + 5} = \frac{95}{110} \approx 0.86$$

Але якщо просто будемо передбачати, що всі листи Не-спам, то отримаємо більш високу акуратність:

TN=100, FP=0, TP=0, FN=10

$$Accuracy = \frac{0 + 100}{0 + 100 + 0 + 10} = \frac{100}{110} \approx \mathbf{0.91}$$

Хоча акуратність є швидким та інформативним індикатором продуктивності моделі, для незбалансованого набору даних – це не кращій вибір.

Деякі метрики

2. Точність (**Precision**).
Точністю називається частка правильних відповідей моделі в межах класу - відношення кількості правильних позитивних прогнозів до всіх позитивних класифікацій.

$$Precision = \frac{TP}{TP + FP}$$

Перевіримо для попереднього прикладу:

100 НЕ-спам листів, 90 з яких класифікатор визначив вірно (TN=90, FP= 10), і 10 спам-листів, 5 з яких класифікатор також визначив вірно (TP = 5, FN = 5).

$$Precision = \frac{5}{5 + 10} = \frac{5}{15} \approx 0.33$$

Деякі метрики

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

3. Повнота (**Recall**).

Повнота - це частка істинно позитивних класифікацій. Повнота показує, яку частку об'єктів, що реально належать до позитивного класу, передбачено правильно.

Іншими словами: відношення кількості правильних позитивних прогнозів до всіх позитивних зразків.

Перевіримо для попереднього прикладу:

100 НЕ-спам листів, 90 з яких класифікатор визначив вірно (TN=90, FP= 10), і 10 спам-листів, 5 з яких класифікатор також визначив вірно (TP = 5, FN = 5).

$$Recall = \frac{5}{5 + 5} = \frac{5}{10} \approx 0.5$$

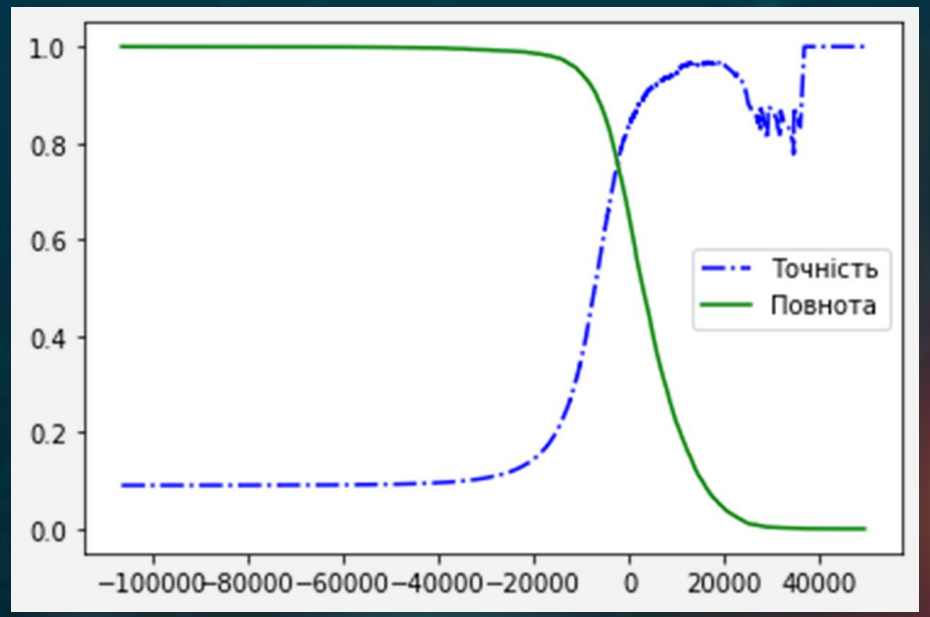
2 3 8 3 | 5 3 5 5 8 3 5 5 5 5 5

TN=4, FP=3, TP=8, FN=0

$$Precision = \frac{8}{8 + 3} = \frac{8}{11} \approx 0.73$$

$$Recall = \frac{8}{8 + 0} = \frac{8}{8} \approx 1$$

Точність vs Повнота



Деякі метрики

4. F1-міра (**F1-score**).

Precision і Recall не залежать від співвідношення класів (на відміну від Accuracy) і тому можуть бути застосовні в умовах незбалансованих вибірок.

Часто в реальній практиці стоїть завдання знайти оптимальний баланс між цими двома метриками.

Тому, потрібна метрика яка об'єднувала б у собі інформацію про точність та повноту алгоритму. Саме такою метрикою є F1-міра.

F1-міра є гармонійним середнім між точністю і повнотою. Вона наближається до нуля, якщо точність або повнота наближаються до нуля.

$$F1\text{-}score = \frac{2 * Precision * Recall}{Precision + Recall}$$

Перевіримо для попереднього прикладу:

100 НЕ-спам листів, 90 з яких класифікатор визначив вірно (TN=90, FP= 10), і 10 спам-листів, 5 з яких класифікатор також визначив вірно (TP = 5, FN = 5).

$$\text{F1-score} = \frac{2*0{,}33*0{,}5}{0{,}33+0{,}5} = \frac{0{,}33}{0{,}83} \approx 0.4$$

Template: AI Tech Agency presentation
Slidesgo